CLOUD & DEVOPS

Prometheus & Grafana Visual Guide

Metrics, PromQL, alerts, dashboards —
the open-source observability stack.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Metrics Model

Four types cover almost everything you measure.

Counter

only goes up: http_requests_total

Gauge

goes up/down: active_connections

Histogram

buckets of observations: latency

Summary

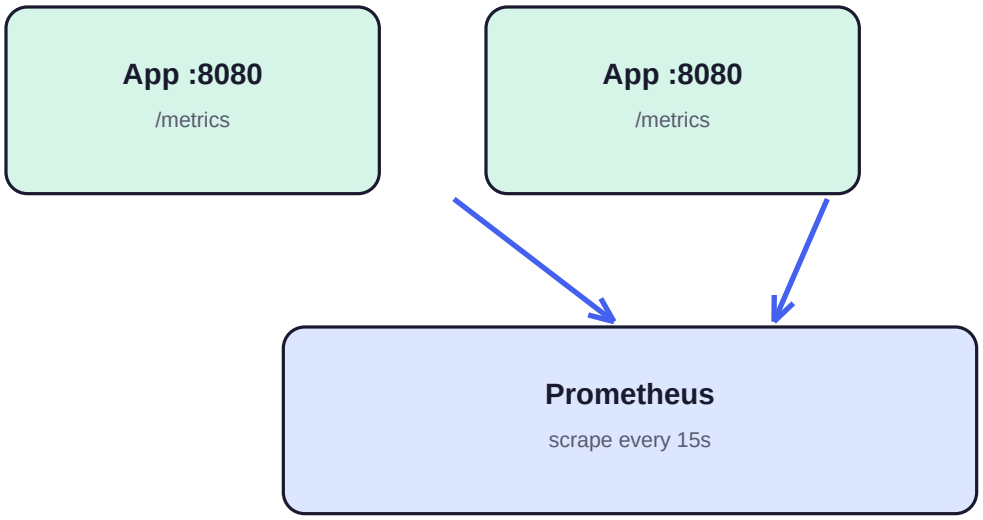
precomputed quantiles: use histograms instead

Labels are dimensions: http_requests_total{method='GET', status='500'} — slice without new metrics.

STEP 2/6

Pull Model: Prometheus Scrapes You

Targets expose /metrics; Prometheus pulls on an interval.



Pull > push for monitoring: dead targets are obvious (no scrape = down), no client buffering.

- Service discovery (K8s, Consul) finds targets automatically — no static config.
- Short-lived jobs push via Pushgateway; everything else gets scraped.
- Scrape interval is your resolution floor: 15s is standard, 5s for hot paths.

STEP 3/6

PromQL Basics

Query language for time series — rates, not raw counters.

rate(http_requests_total[5m])

per-second rate over 5 min — the workhorse query

```
sum by (status) (rate(http_requests_total[5m]))
histogram_quantile(0.99, rate(http_latency_bucket[5m]))
```

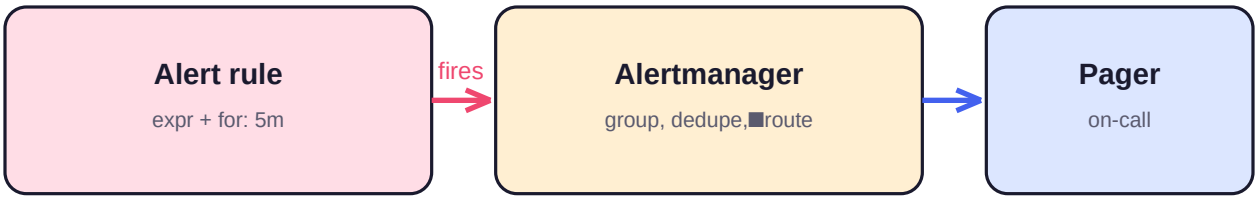
Never graph raw counters — they only go up. rate() turns them into meaningful slopes.

- sum by (label): aggregate across dimensions; without it you get per-pod noise.
- histogram_quantile: p50/p99 from buckets — accurate tail latency.
- offset / time shifts: compare to last week with `offset 1w`.

STEP 4/6

Alerting: Rules + Alertmanager

Prometheus evaluates rules; Alertmanager routes and dedupes.



`for: 5m` means the condition must hold 5 minutes — kills flapping pages.

- Alert on SYMPTOMS (error rate, latency) not causes (CPU) — pages must be actionable.
- Grouping: one incident = one notification, not 50 pod alerts.
- Inhibition: if the region is down, suppress the per-service pages.

STEP 5/6

Dashboard Design (Grafana)

Dashboards answer questions — design top-down.

Row 1: GOLDEN SIGNALS

latency, traffic, errors, saturation

Row 2: SERVICE HEALTH

per-endpoint breakdown, dependencies

Row 3: DEBUG DETAIL

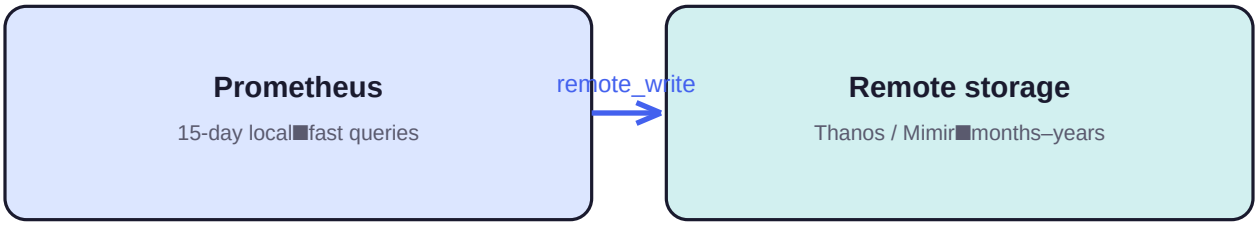
per-pod, per-instance, logs link

Every panel links somewhere: dashboard → logs → traces. A number without a next click is decoration.

STEP 6/6

Retention & Federation

Prometheus is local-first — plan for long-term storage.



Recording rules: precompute expensive queries (e.g. per-service error ratio) into new series.

High cardinality kills: a label with 10k values × 10k series = storage meltdown. Keep label values bounded.

STEP ✓/✓

Key Takeaways

Prometheus & Grafana — Remember

- Four metric types: counters, gauges, histograms (+ summaries) — labels add dimensions.
- Pull model: targets expose /metrics, Prometheus scrapes; dead targets are obvious.
- PromQL: rate() your counters, sum by () to aggregate, histogram_quantile for p99.
- Alert on symptoms with `for:` clauses; Alertmanager groups, dedupes, routes.
- Dashboards top-down: golden signals → service health → debug detail, every panel clickable.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev